สรุปผลการทดสอบซอฟต์แวร์ของโปรเจกต์ ChaoJakk

Lab 08 (นำเครื่องมือไปใช้กับโปรเจกต์ของตัวเอง) และข้อ 2 (ออกแบบ unit test, เทียบโค้ดด้วย V&V, เขียน stub / spy / mock / Faker)

รายวิชา Software Testing (CAMT)
GitHub yanutchara2501 | repo การบ้าน `camt-software-testing` (โฟลเดอร์ `own-project/chaojakk/`)
โปรเจกต์ที่ทดสอบ ChaoJakk (เช่าจักร) ตลาดเช่าเครื่องจักรก่อสร้างพร้อมคนขับ

1. ภาพรวม

ChaoJakk ฝั่ง backend เขียนด้วย Node 24, Express 5, Prisma และ PostgreSQL ส่วน frontend ใช้ Vue 3 กับ TypeScript โปรเจกต์ทดสอบด้วย Vitest และ Supertest อยู่แล้ว (ฝั่ง backend มีไฟล์ test 223 ไฟล์ ก่อนเริ่มงานนี้) เราจึงไม่ได้สร้างระบบทดสอบใหม่ แต่ใช้แนวคิดจาก workshop ตรวจหาช่องว่าง เติม test และหาจุดอ่อนของโค้ดที่ยังไม่มีใครตรวจ

งานทั้งหมดทำใน `git worktree` แยก เพื่อไม่ให้ไปแตะงานที่ยังไม่ได้ commit บน `main` ของโปรเจกต์ โดยมี branch ดังนี้

Branch	ใช้ทำอะไร
`testing-workshop`	Lab 08: test ใหม่ 4 ไฟล์ และเอกสาร
`vv-original`	โค้ดเดิมก่อนทำ V&V (คือ `origin/main` ที่ commit `8da4494d` ไม่ได้แก้อะไร)
`vv-improved`	โค้ดหลังทำ V&V พร้อม unit test ของฟีเจอร์ล็อกอินและ 2FA

ผลลัพธ์โดยย่อ	จำนวน
ไฟล์ test ใหม่ / จำนวน test ใหม่ (ผ่านทั้งหมด)	5 ไฟล์ / 79 test
ช่องโหว่ในโค้ดเดิมที่หาเจอและแก้พร้อมหลักฐาน	3 จุด
ไฟล์ซอร์สที่แก้	1 ไฟล์ (เพิ่ม 32 ลบ 5 บรรทัด)
คะแนน Readiness checklist (เต็ม 24)	17 เป็น 18

2. Lab 08 – นำไปใช้กับโปรเจกต์ของตัวเอง

ขั้นที่ 1: ระบบทดสอบที่มีอยู่แล้ว (harness)

สิ่งที่ต้องมี	มีอยู่ที่ไหน
แยก unit test กับ component test	`platform/api/tests/unit` (88 ไฟล์) และ `tests/feature` (135 ไฟล์)
ฐานข้อมูลจริงสำหรับ component test สร้างจาก migration	`tests/globalSetup.ts` รัน `prisma migrate deploy` บน `TEST_DATABASE_URL` และปฏิเสธการรันถ้าไม่ตั้งค่า (กันไปชนฐานข้อมูลจริง)

ล้างข้อมูลระหว่างไฟล์ test	`tests/helpers/database.ts` และ `fileParallelism: false`
แอปที่สร้างโดยไม่ต้อง `listen()`	`createApp()` ใช้คู่กับ supertest
สร้างข้อมูลทดสอบ	`tests/helpers/factories.ts`
CI	`.github/workflows/ci.yml` มี Postgres service รัน typecheck, test และ build

ขั้นที่ 2: Characterize ก่อนแก้ (จดว่าโค้ดทำอะไรอยู่จริง)

endpoint สำคัญอย่างการจองและการชำระเงินมี feature test ครอบคลุมอยู่แล้ว เราจึงเลือกสิ่งที่ **ทุก route ที่มี `:id` ใช้ร่วมกัน** แต่ยังไม่มี test ตรง ๆ คือ `parseId` (`src/lib/params.ts`) ไฟล์ `tests/unit/params.test.ts` บันทึกพฤติกรรมตามที่เป็นจริง ส่วนที่แปลกติดป้าย `WEIRD` และยังไม่แก้

ค่าที่ใส่	ผลปัจจุบัน
`"42"` , `42`	ได้ `42n`
`0` , `-1` , `1.5` , `"abc"` , ค่าว่าง, `null` , `undefined` , `NaN` , `"Infinity"`	ตอบ 404 "ไม่พบข้อมูล"
WEIRD `"1e3"`	ถูกรับเป็น `1000n`
WEIRD `"0x10"`	ถูกรับเป็น `16n`
WEIRD `" 7 "` (มีช่องว่าง)	ถูกรับเป็น `7n`
WEIRD `"9007199254740993"`	กลายเป็น `9007199254740992n` (เลขเกิน 2^{53} เพี้ยนไปเป็นเลขข้างเคียง)

ข้อสุดท้ายสำคัญที่สุด เพราะคอมเมนต์ในโค้ดสัญญาว่า id ที่ผิดรูปแบบต้องดูเหมือน id ที่ไม่มีอยู่ แต่เลขใหญ่กลับกลายเป็น id อื่นแทน

ขั้นที่ 3: test ใหม่

ไฟล์	ระดับ	test	ตรวจอะไร
`unit/totp.test.ts`	Unit	20	รหัส 2FA กับนาฬิกาที่หยุดไว้: รหัสปัจจุบัน, เร็ว/ช้าไป 1 ช่วง รับ, เกิน 2 ช่วง ปฏิเสธ, ช่องว่างในรหัส, รหัสหรือ secret ผิดรูปแบบต้องคืน `false` ไม่ throw; รหัสกู้คืน 8 รหัส รูปแบบ `XXXXX-XXXXX` ไม่มีตัวอักษรที่หน้าตาคล้ายกัน ไม่ซ้ำ
`unit/geo.test.ts`	Unit	9	ระยะทาง Haversine เทียบกับค่าที่คำนวณจากสูตรเอง (1 องศา = 111,195 ม.), สมมาตร, ขั้วโลกตรงข้าม, กรุงเทพ–เชียงใหม่ประมาณ 580 กม.
`unit/params.test.ts`	Unit (characterization)	17	ตามตารางในขั้นที่ 2

`feature/listingIdParam.test.ts`	Component (supertest + PostgreSQL)	5	`GET /api/v1/listings/:id` ใส่ `abc`, `0`, `-5`, `1.5` ต้องได้สถานะและเนื้อหาตอบเหมือน id ที่ไม่มีอยู่

เราพิสูจน์ว่า test แดงได้จริงโดยแก้ซอร์สชั่วคราวแล้วคืนค่า: เปลี่ยน `window: 1` เป็น `0` ใน `totp.ts` ทำให้ test แดง 2 ข้อ และเปลี่ยนรัศมีโลกใน `geo.ts` ทำให้ test แดง 4 ข้อ

ขั้นที่ 4: Readiness checklist (0 = ไม่มี, 1 = มีบางส่วน, 2 = ครบและอัตโนมัติ)

#	หัวข้อ	รอบ 1	รอบ 2	หลักฐาน
1	ระบุ test boundary ชัดเจน	1	2	มีโฟลเดอร์แยกอยู่แล้วแต่ยังไม่มีเอกสารอธิบาย เอกสารนี้คือส่วนที่ขาด
2	แยก test ที่ไม่ใช้ I/O ออกจากที่ใช้ DB	1	1	แยกโฟลเดอร์แล้ว แต่ `globalSetup` ทำงานกับทุก test จึงต้องรอฐานข้อมูลแม้เป็น unit test
3	รันทั้งหมดด้วยคำสั่งเดียว	1	1	`npm test` แยกกันระหว่าง api กับ frontend และต้องมี `.env` กับ Postgres
4	ฐานข้อมูล test สร้างจาก migration	2	2	`globalSetup.ts`
5	ไม่มีขั้นตอน DB ที่ต้องทำด้วยมือ	1	1	Postgres ตอนพัฒนามาจาก compose file นอก repo
6	เหมือนกันทั้งในเครื่องและ CI	2	2	`ci.yml` เรียก npm script ชุดเดียวกัน
7	ใช้ builder / factory สร้างข้อมูล	2	2	`tests/helpers/factories.ts`
8	test ไม่พึ่งกัน	2	2	ล้างข้อมูลระหว่างไฟล์ และรันทีละไฟล์ (ยังไม่ได้ลองสุ่มลำดับ)
9	ควบคุมเวลาและค่าสุ่มได้	1	1	ใช้ fake timers ใน 5 ไฟล์เท่านั้น และไม่มีตัวสร้างข้อมูลสุ่มแบบ seed คงที่
10	dependency สำคัญ inject ได้	1	1	`createApp()` เป็นจุดแทนที่ได้ แต่ Prisma เป็น singleton จึงเปลี่ยนฐานข้อมูลไม่ได้
11	ใช้ test double ถูกระดับ	2	2	`vi.mock` 11 ไฟล์ และ `vi.spyOn` 4 ไฟล์ จาก 223 ฐานข้อมูลเป็นของจริง
12	ส่วนเสี่ยงที่สุดมี characterization test	1	1	ยังไม่มีแบบชัดเจน `parseId` เป็นชิ้นแรก (เล็ก)
	รวม (เต็ม 24)	**17**	**18**	**คะแนนขยับน้อยเพราะโปรเจกต์มี test ดีอยู่แล้ว ค่าของงานนี้อยู่ที่สิ่งที่ค้นพบด้านล่าง**

สิ่งที่พบและข้อเสนอต่อ

1. **`npm run typecheck` ล้มอยู่แล้วบน `origin/main`** ที่ `tests/unit/labelParts.test.ts` บรรทัด 159–160 อ่าน `hoursCharged` ซึ่งไม่มีใน `RefundBreakdown` CI รัน typecheck ก่อน test จึงน่าจะแดง (เราไม่ได้แก้ไฟล์นี้)
2. **unit test ต้องรอฐานข้อมูล** เสนอแยก `vitest.config.ts` เป็น 2 project (unit ไม่ใช้ `globalSetup`) จะได้รัน unit ได้ในไม่กี่วินาทีโดยไม่ต้องเปิด Docker

3. `parseId` **รับ** `"1e3"` **,** `"0x10"` **และเลขเกิน** 2^{53} **แล้วเพี้ยนเงียบ ๆ** เสนอให้เช็กด้วย `/^[1-9]\d*$/` และ `Number.isSafeInteger` แล้วแก้ test ที่ติดป้าย `WEIRD` พร้อมกัน
4. **ยังไม่มีข้อมูลสุ่มแบบ seed คงที่** เสนอเพิ่ม Faker ใน factories

3. ข้อ 2.1 – ออกแบบ unit test (ฟีเจอร์ล็อกอินและ 2FA)

ทำไมเลือกฟีเจอร์นี้ เป็นด่านแรกของทุกบัญชี ถ้าพลาดคนอื่นอาจเข้าบัญชีแอดมินหรือผู้ให้เช่าได้ อีกทั้งมีตัวช่วย (collaborator) หลายตัวที่ต้องจำลอง คือฐานข้อมูล การตรวจรหัสผ่าน การออก token การบันทึก audit และการตรวจรหัส TOTP จึงใช้ stub, spy, mock ได้ครบ และมีทั้งทางสำเร็จและทางล้มเหลวหลายแบบ

โค้ดที่ทดสอบคือ `platform/api/src/modules/auth/auth.service.ts` (ฟังก์ชัน `login` และ `redeemTwoFactorChallenge`) ไฟล์ test คือ `tests/unit/authLogin.test.ts` มี 28 test

หน่วย	หน้าที่	ตัวช่วยที่จำลอง
`login`	ตรวจรหัสผ่านและสถานะบัญชี แล้วออก token หรือออก challenge ถ้าเปิด 2FA	prisma (stub), `verifyPassword` (stub), `issueToken` (mock), `recordAudit` (spy)
`redeemTwoFactorChallenge`	แลก challenge พร้อมรหัส TOTP หรือ recovery code เป็น token	ตารางจำลองในหน่วยความจำ (fake), `verifyTotp` (stub), `issueToken` (mock)

เทคนิคออกเคส	ใช้ตรงไหน	ได้อะไร
Equivalence partitioning	ตัวระบุตัวตนที่ผู้ใช้พิมพ์: อีเมล, เบอร์ที่มีขีด, เลขบัตร 13 หลักที่มีขีด (UT-L9 ถึง L11)	เลือกตัวแทนต่อกลุ่ม เคสน้อยแต่ครอบคลุม
Boundary value analysis	อายุ challenge 5 นาที (UT-L14, UT-R4 ลองใช้หลังหมด 1 วินาที)	จับ bug ที่ตัวเลขอายุเพี้ยนได้
Decision table	ผลของ login ตามรหัสผ่าน / สถานะบัญชี / การเปิด 2FA	เห็นว่าชุดเงื่อนไขไหนยังไม่มีเคส
State transition	challenge: ใหม่ เป็น ใช้แล้ว หรือหมดอายุ (UT-R1 ถึง R6)	บังคับให้คิดว่า "ใช้ซ้ำ" ต้องเกิดอะไร
Error guessing	ข้อความเหมือนกันทั้งกรณีไม่มีผู้ใช้และรหัสผิด (L6), ตรวจรหัสผ่านแม้ไม่มีผู้ใช้เพื่อให้เวลาตอบเท่ากัน (L7), audit เก็บ hash ไม่เก็บอีเมลจริง (L5)	ล็อกกฎความปลอดภัยที่เดิมมีแต่คอมเมนต์ ไม่มี test
Concurrency test	สองคำขอพร้อมกัน (UT-R6, R7)	พิสูจน์ช่องโหว่แบบ race condition ที่ test ปกติมองไม่เห็น

4. ข้อ 2.2 – เทียบโค้ดเดิมกับโค้ดหลัง V&V

Verification ถามว่า "เขียนถูกตามกฎของตัวเองไหม" ส่วน **Validation** ถามว่า "ตรงกับที่ผู้ใช้ต้องการไหม" เราเทียบด้วยคำสั่ง `git diff vv-original vv-improved --stat -- platform/api/src` ซึ่งมีไฟล์ซอร์สที่เปลี่ยนเพียงไฟล์เดียวคือ `auth.service.ts`

ส่วนที่ไม่เปลี่ยน

- ส่วนเช็กรหัสผ่าน เช็กสถานะบัญชี และออก challenge ใน `login` พฤติกรรมถูกต้องอยู่แล้ว (UT-L1 ถึง L16 ผ่านกับโค้ดเดิม) การเขียนใหม่มีแต่ความเสี่ยง
- `lib/totp.ts` , `lib/password.ts` , `lib/tokens.ts` , router และ schema ไม่พบปัญหา และมี test เดิมคุมอยู่
- ข้อความ error ภาษาไทยทั้งหมด เพราะฝั่ง frontend อ่านข้อความนี้โดยตรง

ส่วนที่เปลี่ยน

#	สิ่งที่เปลี่ยน	V หรือ V	เหตุผล	หลักฐาน
1	`redeemTwoFactorChallenge` ใช้ challenge ด้วย `updateMany({ where: { id, consumedAt: null } })` แล้วตรวจว่าแก้ได้ 1 แถว	Verification	ของเดิมเช็ก `consumedAt` แล้วค่อย `update` เป็นคนละคำสั่ง ถ้าสองคำขอเข้าพร้อมกันจะผ่านทั้งคู่และได้ token สองใบ ทั้งที่ challenge ใช้ได้ครั้งเดียว	UT-R6 แดงกับโค้ดเดิม (ได้ 2 token) เขียวหลังแก้
2	`consumeRecoveryCode` เขียนกลับเฉพาะเมื่อค่าที่เก็บยังเป็นค่าที่เพิ่งอ่านมา (compare-and-swap)	Verification	ของเดิมอ่านรายการ ตัดรหัสออก แล้วเขียนกลับ ถ้าสองคำขอถือรหัสเดียวกันจะอ่านรายการเต็มทั้งคู่ ทำให้ recovery code ใช้ได้มากกว่า 1 ครั้ง	UT-R7 แดงกับโค้ดเดิม เขียวหลังแก้
3	ตรวจ `user.status` ว่า active ก่อนออก token ในขั้น 2FA	Validation	ขั้นรหัสผ่านตรวจสถานะแล้ว แต่ challenge อยู่ได้ 5 นาที ถ้าแอดมินระงับบัญชีระหว่างนั้น ผู้ใช้ยังได้ token ซึ่งขัดกับความตั้งใจของการระงับบัญชี	UT-R8 แดงกับโค้ดเดิม เขียวหลังแก้

ลำดับงานตามหลัก red-green: เขียน UT-R6 ถึง R8 ก่อนแล้วรันกับโค้ดเดิมให้เห็นว่าแดง (commit `14ffec83`) จึงค่อยแก้ซอร์ส (commit `4d0b8b55`) จนเขียว

5. ข้อ 2.3 – ตรวจตามโจทย์

ข้อ	ทำที่
2.3.1 Happy path	UT-L1 (ลูกค้า active รหัสถูก ได้ session และ token), UT-L16, UT-R1
2.3.2 Alternative / failure	UT-L4 (รหัสผิด), UT-L6, UT-L7, UT-L8 (บัญชีถูกระงับ), UT-R2 ถึง R4
2.3.3 Stub	`prismaMock.user.findFirst.mockResolvedValue(...)` และ `verifyPasswordMock.mockResolvedValue(...)` ใน UT-L1, L4, L9 ถึง L12
2.3.4 Spy	`recordAudit` ใน UT-L3, L5, L8 (อ่านสิ่งที่ถูกบันทึกภายหลัง)
2.3.5 Mock	`issueToken` ใน UT-L2 (ต้องถูกเรียก 1 ครั้งด้วย `(user.id, 'chaojakk-pwa', context, role)`) และ `not.toHaveBeenCalled()` ใน UT-L4, L13, R2, R8

2.3.6 Faker	UT-L16 (ลูกค้าสุ่ม 5 ชุด) และข้อมูลผู้ใช้ทุกแถวจาก `aUserRow()` ตั้ง seed คงที่ 20261005 เพื่อให้รันซ้ำได้ผลเดิม

6. สรุปเทคนิคที่ใช้: ใช้อะไร ทำไมเลือก ช่วยการเทสอย่างไร

เทคนิค	ใช้ที่ไหน	เหตุผลที่เลือก	ประโยชน์ต่อการเทส
Stub	prisma, `verifyPassword`	อยากทดสอบแค่ตรรกะล็อกอิน ไม่รอฐานข้อมูลจริงหรือ argon2 ที่ช้า	คุมข้อมูลขาเข้าได้เต็มที่ ทั้งไฟล์รันในไม่กี่สิบมิลลิวินาที
Spy	`recordAudit`	อยากรู้ว่าระบบบันทึกอะไรลง audit โดยไม่ขวางการทำงานจริง	ตรวจผลข้างเคียงที่ไม่ได้ return ได้ เช่น ไม่มีอีเมลจริงหลุดเข้า audit
Mock	`issueToken`	บางเรื่อง "การถูกเรียก" คือตัวพฤติกรรม เช่น ล็อกอินไม่ผ่านต้องไม่ออก token เลย	ล็อกกฎความปลอดภัย ถ้าใครเผลอออก token ผิดทาง test แดงทันที
Fake (ตารางจำลองในหน่วยความจำ)	challenge และ recovery code ใน UT-R	stub ตอบค่าเดิมทุกครั้ง แต่ race condition ต้องมีสถานะที่เปลี่ยนตามเวลา	จำลอง "สองคำขอพร้อมกัน" ได้โดยไม่ต้องมีฐานข้อมูลจริง และผลเหมือนเดิมทุกรอบ
Faker + seed คงที่	ข้อมูลผู้ใช้	ข้อมูลที่เขียนมือมักซ้ำแบบเดิมจนพลาดเคสแปลก	ข้อมูลหลากหลายโดยไม่ต้องเขียนเอง และ test แดงแล้วรันซ้ำได้ข้อมูลเดิม
Fake timers	UT-L14, UT-R4, `totp.test.ts`	อายุ 5 นาทีและรอบ 30 วินาทีของ TOTP ไม่อยากรอจริง	ทดสอบค่าที่ขอบเวลาได้ในเสี้ยววินาที และไม่สุ่มพัง
Characterization test	`params.test.ts`	ไม่รู้ว่ามีใครพึ่งพฤติกรรมแปลกอยู่ จึงจดไว้ก่อนยังไม่แก้	ถ้าวันหลังมีคนแก้ test บอกทันทีว่าพฤติกรรมไหนเปลี่ยน
ค่าคาดหวังคำนวณจากสูตร	`geo.test.ts`	ถ้าเอาผลของโค้ดมาเป็นค่าคาดหวัง test จะเขียวเสมอแม้โค้ดผิด	test ตรวจความถูกต้องจริง
Red-green และลองทำให้ test แดงเอง	UT-R6 ถึง R8, mutation ด้วยมือ	test ที่ไม่เคยแดงอาจไม่ได้ตรวจอะไรเลย	มั่นใจว่าแก้ถูกจุด และ test จับ bug ได้จริง
Verification และ Validation	การหาจุดอ่อนในข้อ 2.2	แยกคำถาม "เขียนถูกไหม" กับ "ตรงที่ผู้ใช้ต้องการไหม"	ได้เหตุผลชัดว่าทำไมแก้แต่ละจุด ไม่ใช่แก้ตามความรู้สึก
Atomic update	`updateMany` ที่มีเงื่อนไขใน WHERE	แก้ race condition ด้วย JavaScript ฝั่งเราไม่ได้ เพราะสองคำขอเห็นข้อมูลเก่าพร้อมกัน	ให้ฐานข้อมูลตัดสิน ทดสอบง่าย: สองคำขอพร้อมกันต้องมีผู้ชนะ 1 คนเสมอ

Component test (supertest + DB จริง)	`listingIdParam.test.ts`	ต้องการพิสูจน์ว่า route แปลง error เป็นคำตอบเดียวกันจริง unit test ตรวจได้แค่ว่าฟังก์ชันthrow	ตรวจสัญญาที่เขียนไว้ในคอมเมนต์ของ `parseId` ผ่าน HTTP จริง

7. วิธีรันและข้อจำกัดที่ทราบ

รันใน worktree ที่ `platform/api` (ต้องเปิด Postgres ที่ใช้เป็นฐานข้อมูลทดสอบ และมีไฟล์ `.env`)

คำสั่ง	ผลที่ได้
`npx vitest run tests/unit/authLogin.test.ts`	28 test ผ่าน (ข้อ 2.1 ถึง 2.3)
`npx vitest run tests/unit/totp.test.ts tests/unit/geo.test.ts tests/unit/params.test.ts tests/feature/listingIdParam.test.ts`	51 test ผ่าน (Lab 08)
`git diff vv-original vv-improved --stat -- platform/api/src`	เห็นว่าซอร์สเปลี่ยนไฟล์เดียว

ข้อจำกัดที่ทราบ (ตามจริง)

- เรา **ไม่ได้รัน test backend ทั้งชุด** (227 ไฟล์) เพราะช้ามาก ไฟล์ `bookingLifecycle.test.ts` ไฟล์เดียวใช้ประมาณ 5 นาที จึงหยุดหลังรันได้ 4 ไฟล์ การตรวจผลกระทบของการแก้จึงอาศัย test เดิมที่เรียกโค้ดส่วนนี้ ได้แก่ `twoFactor`, `oauth`, `auth`, `bookingMaintenance` รวม 101 test ผ่านทั้งหมด
- มี test แดงอยู่ก่อนแล้ว 1 ข้อ คือ `bookingAgreement.test.ts > leaves a short hire as a single payment` เราลองคืน `auth.service.ts` เป็นเวอร์ชันเดิมแล้วรันข้อนี้ ก็แดงเหมือนกัน แปลว่าไม่เกี่ยวกับการแก้ของเรา (ยังไม่ได้หาสาเหตุ)
- `npm run typecheck` ล้มอยู่ก่อนแล้วที่ `labelParts.test.ts` (ดูหัวข้อที่พบในบทที่ 2) ไฟล์ที่เราเพิ่มไม่มีข้อผิดพลาดด้าน type
- เพิ่ม `@faker-js/faker@9.9.0` เป็น devDependency ของ `platform/api` เพื่อใช้ในข้อ 2.3.6

เอกสารฉบับเต็มและโค้ดอยู่ใน repo การบ้าน: `own-project/chaojakk/README.md` (Lab 08), `vv-unit-test-design.md` (ข้อ 2), `tests/` (test ทั้ง 5 ไฟล์) และ `vv-auth.service.diff` (ส่วนต่างของซอร์สที่แก้)